Лабораторная работа №3: конвертер валют через Spec-Driven Development и Google Antigravity

Тип приложения: десктопное или веб-приложение с графическим интерфейсом. Консольное приложение не принимается.

Инструменты: GitHub Spec Kit (подготовка спецификации) + Google Antigravity (сборка)

Репозиторий: новый, отдельный от Лаб 1 и Лаб 2

Цель работы

Пройти полный цикл spec-driven development: сначала описать приложение в виде набора markdown-спецификаций, затем передать эти спецификации агенту другой компании и получить работающий код. Проверяется не умение писать код, а умение формулировать требования так, чтобы по ним собирал агент.

Часть 1. Подготовка инструментов

GitHub Spec Kit. Официальный репозиторий: https://github.com/github/spec-kit

Установку, настройку и знакомство с инструментом студент выполняет самостоятельно по документации проекта. Это часть задания: разобраться в незнакомом инструменте по первоисточнику.

Google Antigravity. Сайт: https://antigravity.google

Важное предупреждение: в мае 2026 года на Google I/O вышла версия 2.0, в которой Antigravity перестал быть форком Visual Studio Code и стал самостоятельным приложением с собственным CLI и SDK. Все обучающие материалы, снятые до мая 2026 года, показывают устаревший интерфейс. Ориентируйтесь на официальную документацию, а не на видео.

Модели. Antigravity — закрытая платформа. Подключить собственные API-ключи или сторонние модели нельзя. Доступен фиксированный список моделей: Gemini 3.5 Flash по умолчанию, опционально Gemini 3.1 Pro, Claude Sonnet 4.6, Claude Opus 4.6, GPT-OSS 120B. Бесплатного тарифа для этой лабораторной достаточно. Платные тарифы увеличивают квоту, но не дают других моделей.

Часть 2. Spec-Driven Development

Выполняется в VS Code с GitHub Copilot — окружение уже настроено после Лаб 1.

Порядок работы соответствует документации Spec Kit: инициализация проекта командой `specify init` с интеграцией copilot, затем последовательность слэш-команд в чате Copilot. Ключевые этапы:

1. Конституция проекта — принципы и ограничения
2. Спецификация — что делает приложение
3. План — как оно устроено технически

4. Задачи — разбиение на шаги реализации
5. Проверка согласованности артефактов перед реализацией

Точные имена команд и их порядок берутся из актуальной документации Spec Kit, а не из этого документа: инструмент активно развивается, команды переименовывались.

Результат части 2: сгенерированные markdown-файлы закоммичены в репозиторий. Реализация на этом этапе ещё не начата.

Часть 3. Сборка в Antigravity

Репозиторий с готовыми спецификациями открывается в Antigravity. Реализация выполняется агентом Antigravity на основе файла задач, подготовленного в части 2.

Ручное написание кода не запрещено, но каждое место, где пришлось вмешаться вручную, фиксируется в отчёте.

Ключевая идея этапа: артефакты Spec Kit — обычные markdown-файлы в репозитории. Спецификация, написанная для одного агента, исполняется другим. Это и проверяется.

Часть 4. Требования к приложению

Назначение. Конвертер валют с графическим интерфейсом.

Источник данных — на выбор студента:

- Любой бесплатный API курсов валют
- Парсинг сайта, например официального сайта Национального банка Молдовы

Оба варианта равноценны. Выбор обосновывается в отчёте.

Справочно: BNM отдаёт данные в формате XML по адресу вида `https://www.bnm.md/en/official_exchange_rates?get_xml=1&date=DD.MM.YYYY`. JSON-эндпоинта нет. Официальный курс публикуется в рабочие дни, обычно до 13:00 по времени Кишинёва, и действует весь следующий рабочий день.

Интерфейс:

1. Поле ввода суммы
2. Выбор исходной валюты
3. Выбор целевой валюты
4. Кнопка конвертации, неактивная до заполнения обязательных полей
5. Отображение результата
6. Отображение даты курса и источника данных

Обязательное поведение:

- **Нет сети.** Приложение не падает, выводит понятное сообщение, при наличии сохранённых данных предлагает работать с последним успешно полученным курсом с указанием его даты.

- **Выходные и праздники.** Если источник не публикует курс в выбранную дату, приложение корректно обрабатывает пустой ответ и берёт последний доступный курс, явно сообщая об этом пользователю.
- **Некорректный ввод.** Отрицательная сумма, ноль, буквы, пустое поле — приложение сообщает об ошибке и не выполняет конвертацию.
- **Одинаковые валюты.** Выбор одной и той же валюты в обоих списках обрабатывается без ошибки.
- **Кэширование.** Последний успешный ответ источника сохраняется локально и переживает перезапуск приложения.

Часть 5. Работа с Git

Требования наследуются из Лаб 1 и Лаб 2 и ужесточаются.

- Отдельная ветка на каждый этап работы, не одна ветка на всю лабораторную
- Минимум: ветка для спецификаций, ветка для реализации, ветка для тестов
- Осмысленные сообщения коммитов, отражающие суть изменения
- История коммитов должна показывать процесс: один коммит «всё готово» не принимается
- Слияние в основную ветку через Pull Request, не прямым push

Часть 6. Тесты

Обязательное покрытие unit-тестами:

- Разбор ответа источника данных (API или разметки сайта)
- Расчёт конвертации, включая обратное направление
- Поведение при пустом или некорректном ответе источника
- Валидация пользовательского ввода

Тесты запускаются локально одной командой. Команда запуска описана в README.

Часть 7. Отчёт

Файл `REPORT.md` в корне репозитория:

1. Какой источник данных выбран и почему
2. Какие модели использовались в Antigravity на каких этапах
3. Что агент собрал корректно с первой попытки
4. Где агент ошибся, и что пришлось исправлять вручную
5. Насколько итоговый код соответствует спецификации, написанной в части 2

Пункт 4 обязателен к заполнению. Отчёт без единого расхождения между спецификацией и результатом вызывает сомнения в том, что работа выполнялась.

Часть 8. Критерии приёмки

1. Репозиторий новый, отдельный, содержит артефакты Spec Kit
2. Спецификации закоммичены раньше, чем реализация — проверяется по истории git

3. Приложение имеет графический интерфейс
4. Конвертация выполняется корректно в обе стороны
5. Отображаются дата курса и источник данных
6. Отключение сети не роняет приложение
7. Некорректный ввод обрабатывается
8. Кэшированный курс переживает перезапуск
9. Unit-тесты присутствуют и проходят
10. История git отражает процесс, слияние выполнено через Pull Request
11. `REPORT.md` заполнен, включая раздел об ошибках агента